Techniques web en python

Cours 4 : Pandas + Monitorer ses programmes

Rappel

Un petit aperçu du calendrier

SEM	MARDI	PROGRAMME
39	22/09/2026	Intro et le file system
40	29/09/2026	Pythonismes + Qu'est ce qu'internet
41	06/10/2026	Web crawling and scrapping
42	13/10/2026	Intro datascience + Monitoring
43	20/10/2026	Data visualisation + la récursivité
44	*27/10/2026*	*Vacances* **Rendu TP1**
45	**03/11/2026**	**Contrôle 1**

SEM	MARDI	PROGRAMME
46	10/11/2026	Moteurs de recherche + décorateurs **Rendu TP2**
47	17/11/2026	La programmation asynchrone
48	24/11/2026	Les API
49	01/12/2026	Intro to AI Agents **Rendu TP3**
50	**08/12/2026**	**Contrôle 2**
51	15/12/2026	Corrigé + le point à faire
52	22/12/2026	*Pas de cours* **Rendu final TP4**

Seven different symbols represent Roman numerals with the following values:

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Roman numerals are formed by appending the conversions of decimal place values from highest to lowest. Converting a decimal place value into a Roman numeral has the following rules:

- If the value does not start with 4 or 9, select the symbol of the maximal value that can be subtracted from the input, append that symbol to the result, subtract its value, and convert the remainder to a Roman numeral.
- If the value starts with 4 or 9 use the **subtractive form** representing one symbol subtracted from the following symbol, for example, 4 is 1 (`I`) less than 5 (`V`): `IV` and 9 is 1 (`I`) less than 10 (`X`): `IX`. Only the following subtractive forms are used: 4 (`IV`), 9 (`IX`), 40 (`XL`), 90 (`XC`), 400 (`CD`) and 900 (`CM`).
- Only powers of 10 (`I`, `X`, `C`, `M`) can be appended consecutively at most 3 times to represent multiples of 10. You cannot append 5 (`V`), 50 (`L`), or 500 (`D`) multiple times. If you need to append a symbol 4 times use the **subtractive form**.

Given an integer, convert it to a Roman numeral.

Capture d’écra
leetcode

1) tradeof entre la mémoire prise et la complexité du code

```python
def intToRoman( num):
        ones = ["", "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX"]
        tens = ["", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX", "XC"]
        hrns = ["", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC", "CM"]
        ths = ["", "M", "MM", "MMM"]

        return ths[num // 1000] + hrns[(num % 1000) // 100] + tens[(num % 100) // 10] + ones[num % 10]

```

2) stocker moins de valeurs

```python
def intToRoman( num: int) -> str:
        value_to_symbol = {
            1000: 'M', 900: 'CM', 500: 'D', 400: 'CD',
            100: 'C', 90: 'XC', 50: 'L', 40: 'XL',
            10: 'X', 9: 'IX', 5: 'V', 4: 'IV', 1: 'I'
        }

        roman = []
        for value in sorted(value_to_symbol.keys(), reverse=True):

            while num >= value:
                roman.append(value_to_symbol[value])
                num -= value

        return ''.join(roman)
```

3) Une solution générique mais un peu complexe

```python
def intToRoman( num):

        roman_string = []

        sign_to_unit = {"V": "I", "X": "I", "L": "X", "C": "X", "D": "C", "M": "C"}

        value_to_sign = {1000: 'M', 500: 'D', 100: 'C', 50: 'L', 10: 'X', 5: 'V', 1: 'I'}

        for step in sorted(value_to_sign.keys(), reverse=True):

            roman_letter = value_to_sign[step]
            division = num // step
            if division > 0:
                roman_string.extend([roman_letter] * division)
                num -= division * step

            reste_dizaine_suivante = int(str(num % step)[0]) if len(str(num)) == len(str(step)) -1 else 0

            if reste_dizaine_suivante == 4 and int(str(step)[0])!= 5  :
                    num += step / 10
                    roman_string.append(sign_to_unit[roman_letter])

            if reste_dizaine_suivante == 9 and int(str(step)[0])!= 5  :
                    num -= 9* step/10
                    roman_string.append(sign_to_unit[roman_letter])
                    roman_string.append(roman_letter)

        return "".join(roman_string)
```



Pandas

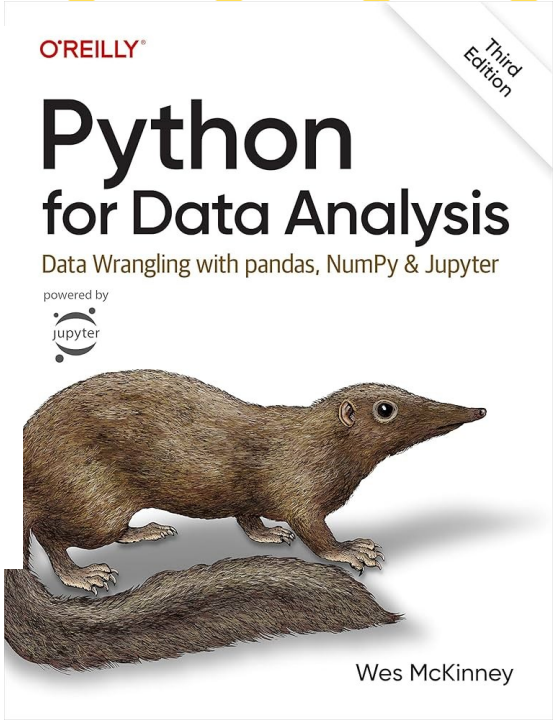


pandas

$$y_i t = \beta' x_{it} + \mu_i + \epsilon_{it}$$



	BandName	WavelengthMax	WavelengthMin
0	CoastalAerosol	450	430
1	Blue	510	450
2	Green	590	530
3	Red	670	640
4	NearInfrared	880	850
5	ShortWaveInfrared_1	1650	1570
6	ShortWaveInfrared_2	2290	2110
7	Cirrus	1380	1360



Pandas : la librairie spécialisée pour traiter et exploiter les donneés

- Un objet fondamental, le DataFrame qui est fait pour stocker des données tabulaires
- Des opérations qui transforment un dataframe en un autre
- Une philosophie reposant sur l'enchaînement des méthodes
- Des manières de combiner plusieurs sources de données un peu à la manière de SQL

  => moins on en fait mieux on se porte, un code court est sans doute bon et la plupart de ce que vous voulez faire peut se faire en trois lignes max

0) L’objet Dataframe

- Une ligne = une observation
- Une colonne =un attribut

équivalent à une liste de dictionnaires

	Nom	Mathématiques	Physique	Informatique	Anglais
0	Alice	15	13	18	14
1	Bob	12	14	16	13
2	Charlie	9	11	12	15
3	David	14	10	15	12
4	Emma	16	17	19	16

1) charger ses données

Import pandas as pd

df = pd.read_csv(path) # tout un écosystème de connecteurs

ou : créer à partir d’un dictionnaire de listes

df = DataFrame(dic)

2) regarder ses données

- df.head(n) / df.tail(n) : afficher les n premières ou dernières lignes
- df[‘clé’] : sélection d’une colonne uniquement

  NB : on peut sélectionner plusieurs colonnes en donnant une liste de clés df[[‘clé1’,’clé2’...]
- df.loc[slicing] : sélection d’une partie des colonnes

  NB : rappel sur le slicing https://www.w3schools.com/python/python_strings_slicing.asp
-

3) masques et sélection d’une partie du df

On peut ne retenir qu’une partie du dataset (lignes) qui satisfont une certaine condition.

Pour se faire on crée ce que l’on appelle **des masques**, des expressions boléennes et on sélectionne la partie du dataframe pour lequel le masque vaut True

même principe que les matrices numpy

syntaxe : df[masque]

df_notes[df_notes['Informatique'] > 15]

4) application d’opérations basiques sur les colonnes

- Une fois qu’on a sélectionné une colonne, on peut effectuer dessus des opérations élémentaires.

  Il faut distinguer les opérations non transformatives (calcul d’une valeur à partir de toutes les valeurs de la colonne) d’opérations transformatives qui modifient le contenu de la cellule (opération appliquée à chaque case).

- Parmi les opérations les plus fréquentes, on trouve .mean()

  .unique() , .count() , .std() (standard deviation = écart type)

5) opérations transformatives

On peut créer de nouvelles colonnes à partir des précédentes

ex df[‘full_price’] = df[‘object_price’] +df[‘tax’]

Quand les colonnes contiennent du texte il faut convertir le contenu en texte avec la méthode .str avant d’effectuer des opérations classiques sur les strings.

- Ex : df[‘prenom’] = df[‘name’].str.split().str[0]
- Exemple de chaînage de méthodes

6) groupby

- Le groupby en pandas permet de regrouper les données d'un DataFrame en fonction des valeurs d'une ou plusieurs colonnes. Ensuite, des fonctions d'agrégation (comme la moyenne, la somme, le comptage, etc.) ou des fonctions personnalisées peuvent être appliquées sur chaque groupe. Cela permet de résumer ou d'analyser les données par sous-ensemble de manière efficace.
- Fait de la (ou des) colonnes données en objet du groupby les observations

  NB : groupby crée un nouveau DataFrame sans modifier le premier !!
-

  grouped = df.groupby('Classe').mean()

7) value counts

- La méthode value_counts() en pandas est utilisée pour compter la fréquence d'apparition des différentes valeurs dans une colonne d'un DataFrame ou d'une Series. C'est un outil très pratique pour obtenir un aperçu rapide de la répartition des données dans une colonne.
- NB : value_counts() renvoie une Series (et non un DataFrame) avec, comme index, les valeurs uniques de la colonne choisie, et comme valeurs, le nombre d'apparitions de ces éléments dans le dataset.
- classe_counts = df['Classe'].value_counts().

8) Gestion des Nan

Nan = Not a Number cf numpy

valeur par défaut lorsqu'une case du tableau est vide.

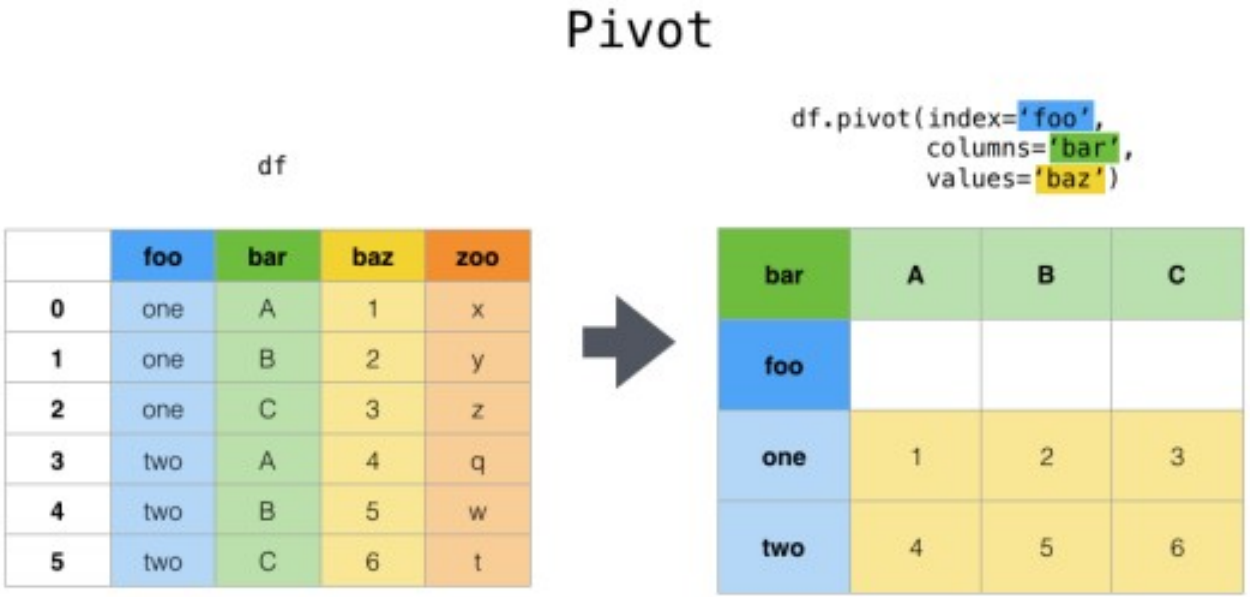
Normalement ces colonnes ne sont pas prises en compte dans les calculs mais peut causer des troubles.

- df.dropnan() : élimine les observations contenant des Nan
- df_nan = df[df.isna().any(axis=1)] filtre pour observer
- df.fillnan('blabla') permet de remplacer les nan par une valeur. Ex « pas de texte disponible » pour mieux comprendre le dataset

9) Les pivots

- Le pivot en pandas est une méthode qui permet de réorganiser ou restructurer un DataFrame en réaffectant les colonnes et les index. Il est particulièrement utile pour transformer des données de forme longue (long format) en données de forme large (wide format), en créant une table pivotée où les valeurs d’une colonne deviennent des colonnes elles-mêmes.
-
- Explication détaillée :
- La méthode pivot() en pandas prend une table de données et "pivot" les lignes en colonnes. Elle permet de réorganiser le DataFrame en fonction de trois éléments :
- index : les lignes sur lesquelles les données sont organisées.
- columns : les colonnes de données à créer à partir des valeurs d'une colonne spécifique.
- values : les valeurs à remplir dans le tableau pivoté (correspondent aux données que l'on souhaite analyser).

- Cf documentation officielle de Pandas



df

	foo	bar	baz	zoo
0	one	A	1	x
1	one	B	2	y
2	one	C	3	z
3	two	A	4	q
4	two	B	5	w
5	two	C	6	t

bar	A	B	C
foo			
one	1	2	3
two	4	5	6

Fusionner plusieurs df : merge vs concat

- Merge : fusionner deux df sur deux propriétés ;
  « fusion imparfaite »

- Concat : coller deux df similaires
  « fusion verticale ou horizontale»

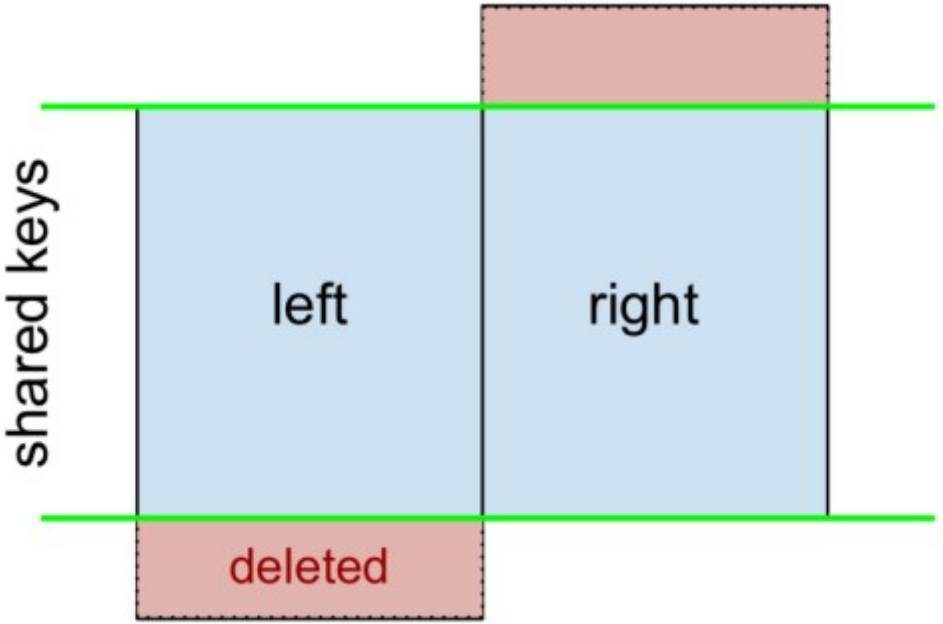
1. `merge` :

La méthode `merge()` est utilisée pour fusionner deux DataFrames en se basant sur une ou plusieurs colonnes communes, comme dans une jointure SQL (JOIN). C'est un moyen d'associer des informations en utilisant des clés correspondantes entre les DataFrames.

- **Clé de fusion** : Il faut souvent spécifier sur quelle(s) colonne(s) (ou clé) se fait la fusion.
- **Type de jointure** : Vous pouvez choisir le type de jointure (inner, outer, left, right), comme en SQL.

Plusieurs types de jointures

Inner join

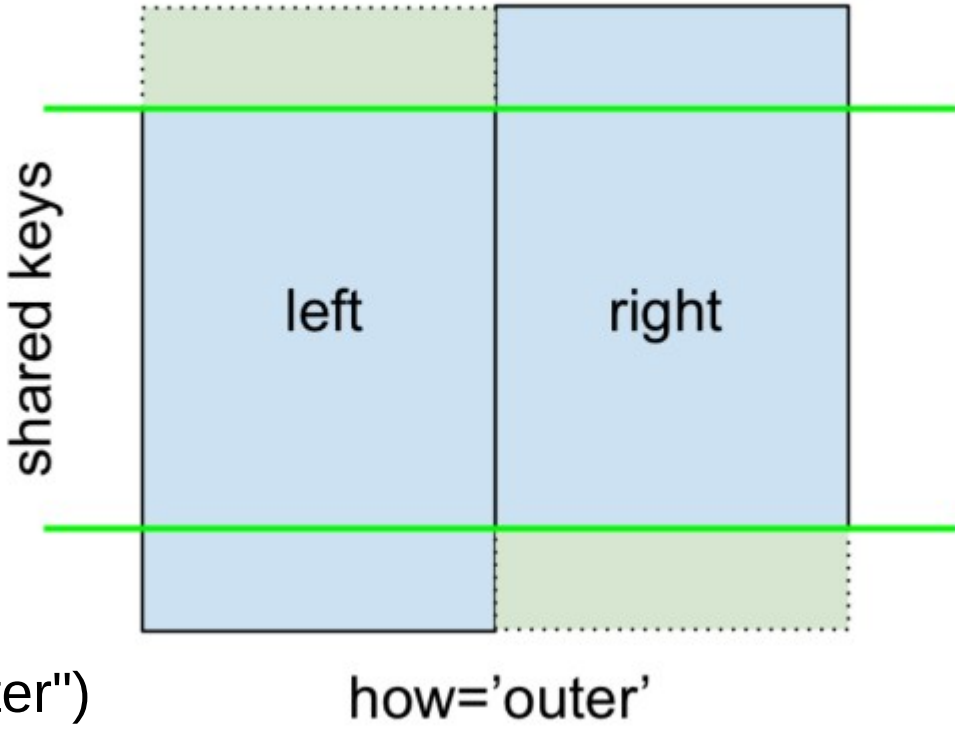


-
-
-
-
-
-
- pd.merge(df1, df2, on= keys, how="inner")

Plusieurs types de jointures

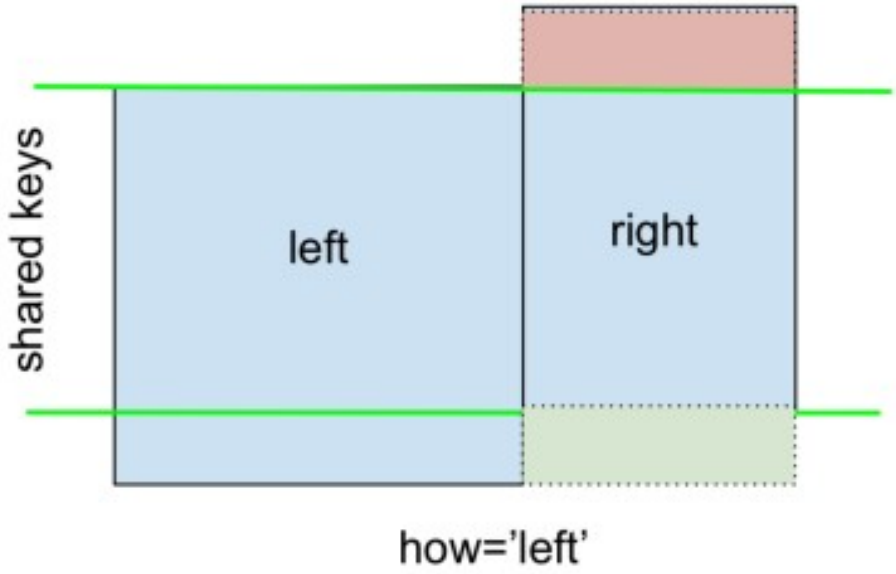
Outer join

-
-
-
-
-
-
- pd.merge(df1, df2, on= keys, how="outer")



Plusieurs types de jointures

left join



- pd.merge(df1, df2, on= keys, how="left")

2. `concat` :

La méthode `concat()` est utilisée pour concaténer (empiler) des DataFrames soit verticalement (par défaut, `axis=0`), soit horizontalement (`axis=1`). Contrairement à `merge`, `concat` ne fait pas de correspondance entre des colonnes spécifiques. Il "colle" simplement les DataFrames ensemble.

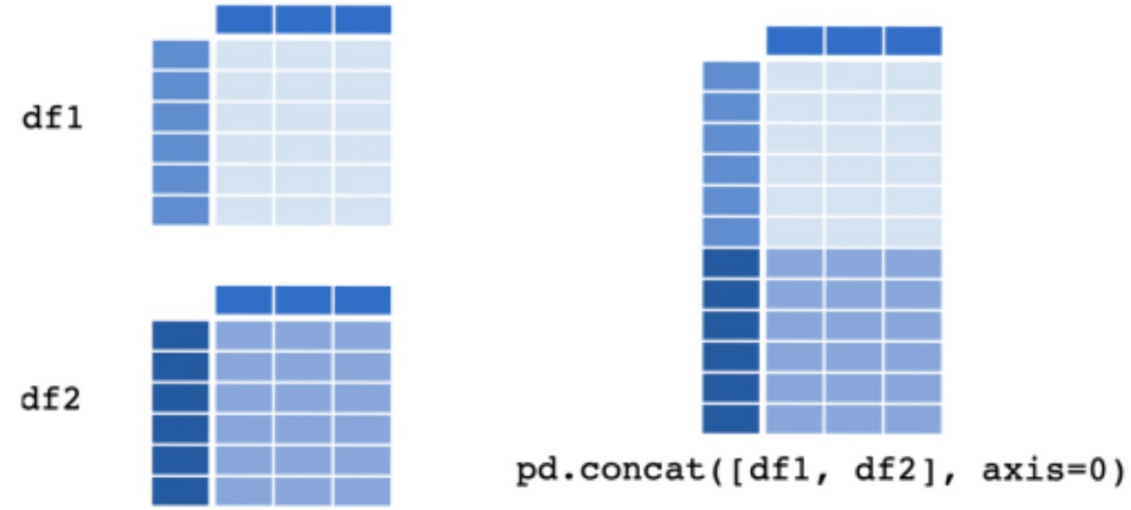
- **Verticalement** : Ajoute les lignes de l'un sous l'autre (doit avoir les mêmes colonnes).
- **Horizontalement** : Ajoute les colonnes côte à côte (les index doivent correspondre ou être gérés).

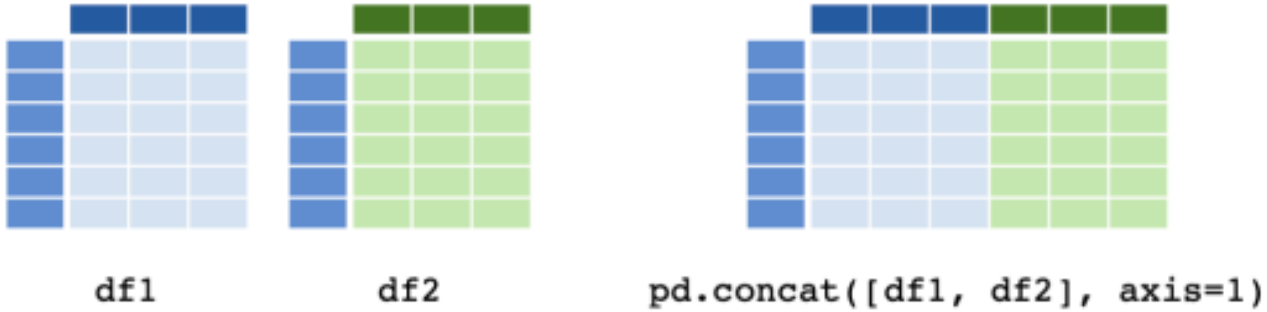
Exemple :

python Copier le code

```python
pd.concat([df1, df2], axis=0)  # Ajoute des lignes
pd.concat([df1, df2], axis=1)  # Ajoute des colonnes
```

- **Utilisation** : Lorsque vous voulez empiler des DataFrames sans correspondance de clé (par exemple, ajouter des données supplémentaires ou combiner différentes mesures).


df1
df2
pd.concat([df1, df2], axis=0)


df1
df2
pd.concat([df1, df2], axis=1)

Bilan

- Beaucoup de possibilités
- Beaucoup de flexibilité

  mais il faut
- Beaucoup pratiquer

  => la librairie phare de la data science

  MAIS pas forcément optimisé pour le traitement de millions de colonnes
  (nouveaux outils comme paquet moins riches mais plus rapides)

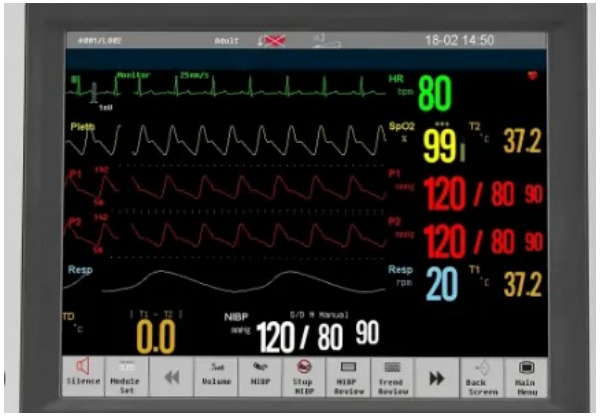


Monitorer son programme

Monitorer son programme

- Le fameux print(« here ») pour trouver l'origine des bugs de son code a ses limites. (programme énorme, milliers de fichiers traités en parallèle…) C'est pourquoi il existe d'autres techniques de monitoring intéressantes que l'on va étudier
- Monitorer c'est surveiller le programme et son exécution (qu'il s 'exécute bien, mais aussi se comporte correctement, soit rapide)
- Destiné à la fois au pogrammeur mais aussi à l'utilisateur pour comprendre ce qui se passe dans la boîte noire qu'est le programme

Python : les bonnes pratiques



1) Mesurer le temps d’exécution

2) Afficher des informations claires pour l’utilisateur :

   le logging et les barres de progression

3) la gestion des erreurs et des exceptions

1) mesurer le temps d’exécution

- Important pour améliorer un programme en production.
- Aussi repérer les problèmes : lenteurs dans le programme = suspect
- Permet de repérer quelles sections du code posent problème => remplacer une fonction en O(n3) par une fonction en O(n2) quand on a beaucoup de données peut diviser par 10 le temps !

  Il faut donc des outils pour repérer les parties lentes du code (bottleneck, goulot d’étranglement) et travailler ces sections

1) mesurer le temps d’exécution

- Python propose une solution simple nommé timers pour monitorer le temps d’exécution de parties de son code

```python
import time

def compter_la_classe(eleves):
    # Début du chronométrage
    debut = time.perf_counter()

    # Logique de la fonction : compter le nombre d'élèves
    nombre_eleves = len(eleves)
    print(f"Nombre d'élèves : {nombre_eleves}")

    # Fin du chronométrage
    fin = time.perf_counter()

    # Calcul du temps d'exécution
    temps_execution = fin - debut
    print(f"Temps d'exécution : {temps_execution:.6f} secondes")
    return nombre_eleves
```

1) mesurer le temps d’exécution

Problème : c’est long, il faut 3 lignes pour mesurer le temps d’exécution d’une seule fonction, et trouver comment stocker les valeurs si on le fait sur tout un code …

- On peut pour ça créer une classe Timer et des méthodes timer.start() timer.stop() qui feront ça de façon plus souple
- On peut créer une mémoire dans l’objet (dic) qui permette de stocker le nom de la fonction lancée

https://realpython.com/python-timer/ pour des idées d’implémentation

1) mesurer le temps d’exécution

Python got us covered

- python -m pip install codetiming

- from codetiming import Timer soit start et stop, soit context managre
- with Timer():
- 	compter_la_classe(eleves)
- Usage comme décorateur @Timer(name="decorator")

1) mesurer le temps d’exécution de son programme

- Un timer efficace est très utile pour monitorer la performance de chaque fonction quand on la code

  Mais parfois quand on travaille sur des programmes intenses, les petits delta sur des fonctions ont un grand impact, un timer classique ne repère pas les petites différences efficacement : il faut lancer la fonction des milliers de fois. Voir le module timeit

- Problème : quand le code devient vraiment compliqué, que l’on appelle certaines fonctions en boucle … même un bon timer ne permet pas de stocker toutes les informations de façon lisible

1) mesurer le temps d’exécution d’un programme

- Dans ce cas on parle plutôt de profiling du programme et peut utiliser des outils adaptés

  python -m cProfile -o nom_du_fichier

  souvent des outils en ligne de commande qui exécutent un code et affichent des statistiques sur le temps de chaque fonciton.
- Je recommande *snakeviz* pour des graphiques sympathiques

- Existe aussi pour faire du profiling de mémoire

2) afficher des informations claires pour l’utilisateur

- Logging : mémoriser des informations durant l’exécution d’un programme.
- Ça peut être la valeur d’une variable, les fonctions utilisées, le nombre d’appels, la taille d’un fichier , l’IP de l’utilisateur…
- Ensuite ces informations peuvent être affichées ou exportées

2) a) le logging

- import logging python a une solution plusieurs niveaux de sévérité

`DEBUG`	`logging.debug()`	Provides detailed information that's valuable to you as a developer.
`INFO`	`logging.info()`	Provides general information about what's going on with your program.
`WARNING`	`logging.warning()`	Indicates that there's something you should look into.
`ERROR`	`logging.error()`	Alerts you to an unexpected problem that's occured in your program.
`CRITICAL`	`logging.critical()`	Tells you that a serious error has occurred and may have crashed your app.

2) a) le logging

Modifier la configuration de base

logging.basicConfig(level=logging.DEBUG)

cet échelonnage en niveaux permet ensuite de définir quelles informations afficher à l'écran sans changer le travail de logging dans le code (le développeur affiche plus d info que pour l utilisateur…)

```
logging.basicConfig(
    filename="app.log",
    encoding="utf-8",
    filemode="a",
    format="{asctime} - {levelname} - {message}",
    style="{",
    datefmt="%Y-%m-%d %H:%M",
)
```

permet aussi de sauver les logs dans un fichier

2) a) le logging

La bonne pratique :

définir le bon niveau d’information pour chaque information affichée

* utiliser les self-documenting expressions pour stocker la valeur de certaines variables critiques

```
logging.debug("variable=%s", variable)
```

souvent pour des gros projets, les équipes définissent leurs loggers custom

2) b) les barres de progression

- Parfois quand on lance des boucles for avec des milliers d’appels et que le code est long, cela rassurerait l’utilisateur de savoir où on en est. Pour cela on peut afficher des barres de progression quand on fait des boucles

- Pour cela, python a tqdm
- cf (taqadum, تـــقدّم), progrès
- (te quiero demasiado).

2) b) les barres de progression

- pip install tqdm

  from tqdm(.auto) import tqdm

-

```
from tqdm import tqdm
    for i in tqdm(range(1000)):
    time.sleep(0.01)
```

  puis on wrappe l'itérable parcouru par tqdm. Marche dans les boucles for, les dicos, les listes par compréhension…

  paramètre desc, préciser la taille de l'objet...

2) b) les barres de progression

- Parfois dans la vie on a des cas plus complexes :
- Nested loops
- Programmes asynchrones
- On veut jouer avec pandas …

  et on aimerait quand même n’avoir qu’une seule barre de progrès pour tout

  La solution est de ne créer qu’une barre et de manuellement incrémenter la barre de progrès quand il faut

  Je n’en dis pas plus et vous laisserai jouer avec dans le TP 3 !

3) la gestion des erreurs

- Normalement un programme python termine dès qu'il rencontre une erreur. Mais ce n'est parfois pas ce qu'on voudrait !

  Ex : pipeline de traitement de données qui prend en entrée des fichiers words. Si qqn glisse par erreur un fichier jpeg il y aura une erreur. Mais on ne veut pas pour autant interrompre la chaîne de traitement et perdre tout le travail déjà fait.

- On peut donc gérer plus finement les erreurs pour préparer son programme à des malfonctions possibles et définir une réponse adaptée

3) la gestion des erreurs

Distinguer erreur vs exception (quand un code correct crashe lors de l’exécution)

Or toutes les erreurs ne sont pas les mêmes : division par 0, fichier n’existe pas, mémoire pleine… python donne plus d’informations sur la raison du crashe en distinguant plusieurs types d’exceptions.

3) gestion des erreurs

- Peut être une opportunité pour mieux contrôler son programme

  ex ; si on repère que le type de fichier est mauvais, inutile d'aller plus loin, on veut s'arrêter là => on *raise* une exception

  if eleve not in liste
- raise Exception(f" {eleve} n'a pas fait le contrôle!")

  print(note[eleve])

  pour généraliser ce genre de checkpoint on utilise le mot clé

  assert suivi d'une expression boléenne
- assert (eleve in liste), f" {eleve} n'a pas fait le contrôle!"

3) gestion des erreurs

- Problème : ça arrête le code. La meilleure manière de faire est d'utiliser un gestionnaire de contexte

  try : le code qu'on veut exécuter mais qui

  va potentiellement crasher

  except : ce qu'on fait si ça ne marche pas (lever

  une exception ou stocker l'erreur)

  else : ce qu'on fait ensuite si le code douteux

  n'a pas crashé

  finally : code exécuté quelle que soit l'issue



Source image; https://medium.com/@ccpythonprogramming/handling-exceptions-in-python-using-try-except-finally-4885e84a9491

3) gestion des erreurs

- On appelle ça l***'exception handling***

  => on prévoit tout ce qui peut aller de travers et on agit en conséquence

  utiliser try / except fait que le programme continue à tourner malgré l'erreur, il faut quand même conserver et traiter l'information que l'erreur a eu lieu.

  On peut raffiner en précisant quel type d'erreur on veut attraper (ex : file not found exception mais si le crash vient d'ailleurs on veut que le programme crashe)

  on peut enchaîner les except avec la façon de réagir à chaque type d'execption

3) gestion des erreurs

La bonne pratique dans un projet est de réfléchir à tout ce qui peut mal se passer dans votre code mais n’est pas de votre responsabilité (hardware, data envoyées…) et prévoir dans les sections critiques une gestion des exceptions prévisibles.

Vous pouvez créer vos propres exceptions particulières à la philosophie de votre programme pour les repérer et les traiter spécifiquement.

Buit-in vs custom exceptions

La gestion des erreurs

```python
class EleveAbsent(Exception):
    """Exception levée lorsqu'un élève est absent de la liste."""
    def __init__(self, nom):
        self.nom = nom
        super().__init__(f"L'élève {self.nom} est absent(e) de la liste.")

def verifier_presence(nom, liste_eleves):
    if nom not in liste_eleves:
        raise EleveAbsent(nom)
    print(f"L'élève {nom} est présent(e).")
```

D’autres sujets pour aller plus loin sur le monitoring

- Parfois avoir des fichiers de logs avec des valeurs n’est pas très interprétable. Il faut donc un moyen de visualiser l’évolution des résultats de son programme sur différents runs

  => création de dashboards. Ex librairie weight and biases

- On peut vouloir vérifier le bon comportement de son code sur des exemples canoniques pour être sur qu’une misa à jour de le détraque pas

  => création de fichiers de unit tests

Pour aller plus loin

- https://realpython.com/python-logging/
- https://www.datacamp.com/tutorial/tqdm-python
- https://realpython.com/python-exceptions/
-